

CHAPTER 4. SYSTEM IMPLEMENTATION

4.1 Implementation Overview

CareerFit is implemented as two application projects and a shared runtime environment. The backend is a Java 21 application based on Spring Boot 3.2.5. The frontend is a React 18 single-page application written in TypeScript and built with Vite. PostgreSQL is the transactional database, Flyway owns schema migration, and Docker Compose provides the reproducible local database and optional backend container. The implementation follows the modular-monolith design established in Chapter 3: domain packages share one backend process while retaining controllers, services, repositories, entities, and DTOs for each functional area.

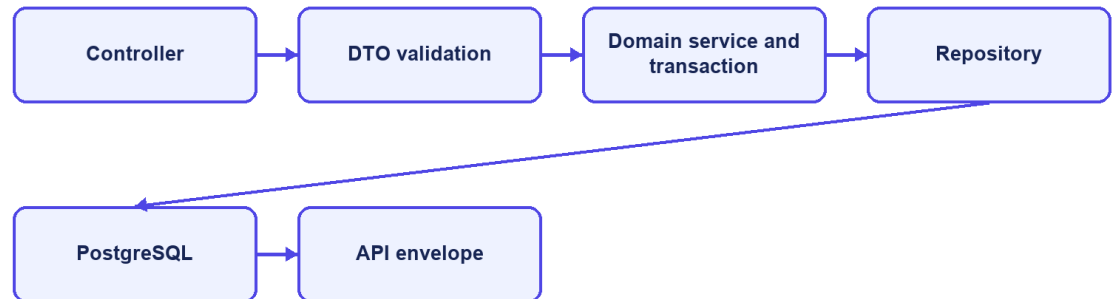
Table 4.1. Main implementation technologies

Layer	Technology	Implementation purpose
Backend language/runtime	Java 21	Domain logic, API, background processing, and integrations
Application framework	Spring Boot 3.2.5	Dependency injection, web MVC, configuration, scheduling, and application lifecycle
Security	Spring Security and JWT 0.12.5	Stateless bearer authentication, role authorization, and JWT handling
Persistence	Spring Data JPA, PostgreSQL 16, Flyway	Entity persistence, queries, constraints, indexes, and versioned migrations
Document processing	PDFBox 3.0.2, Apache POI, Tesseract CLI	PDF/DOCX extraction and OCR fallback for images or scanned PDFs
API documentation	springdoc-openapi 2.5.0	OpenAPI documents and Swagger UI
Monitoring	Spring Actuator, Micrometer Prometheus registry	Health and HTTP/application metrics
Frontend	React 18.3, TypeScript 5.9, React Router 7, React Query 5	Role-based pages, typed API integration, navigation, and server-state queries
Visualization and UI	Recharts and Lucide React	Charts and interface icons
Build and test	Maven, npm/Vite, JUnit, Testcontainers, Playwright	Compilation, automated backend tests, integration environment, and browser E2E tests

The backend package root is `com.careerfit.backend`. Domain packages include `auth`, `candidate`, `cv`, `job`, `employer`, `matching`, `recommendation`, `application`, `feedback`, `automation`, `notification`, `analytics`, `admin`, `audit`, and `settings`. Shared response models,

exceptions, text utilities, validation, security filters, and configuration are placed in common or configuration packages. This structure makes the dependency direction visible without adding network boundaries between modules.

Backend request path



CareerFit IT AutoPilot — thesis diagram

Figure 4.1. Backend module structure and request flow

4.2 Authentication and Authorization Implementation

4.2.1 Login and JWT Processing

AuthController exposes registration, password login, passwordless request/verification, and current-account operations. AuthService validates account state and credentials and delegates token creation to JwtService. Passwords are stored with BCryptPasswordEncoder. A successful login returns an access token and a user DTO containing the ID, email, full name, role, and verification state.

Each authenticated request passes through JwtAuthenticationFilter, a OncePerRequestFilter. The filter requires the Bearer prefix when an Authorization header is present, validates token structure and signature, extracts subject and role, rejects roles outside Candidate, Recruiter, and Administrator, and reloads the account from the database. Reloading prevents a cryptographically valid token from continuing to authorize a deleted or suspended account. The filter then creates a Spring Security authentication object with the corresponding ROLE_* authority.

UserIdResolutionFilter runs after JWT authentication. It resolves the persisted account ID from the authenticated email and stores the UUID as the userId request

attribute. Controllers can therefore pass a stable identifier to services without accepting a user ID supplied by the browser. The implementation currently performs a user lookup in both filters; this is straightforward but creates duplicate database reads that could be consolidated later.

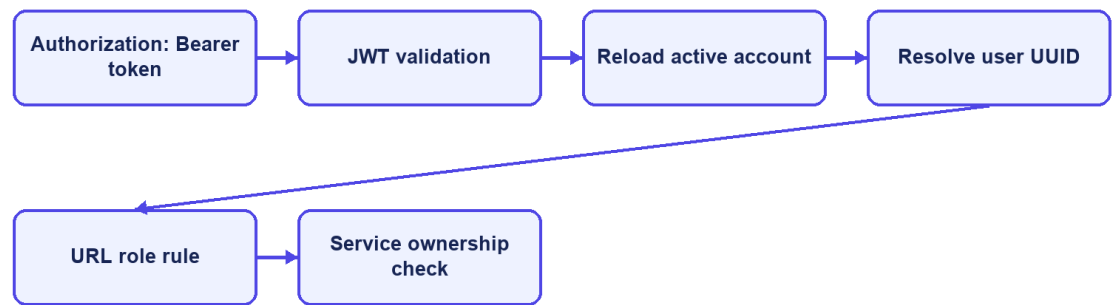
4.2.2 URL Rules and Ownership Checks

SecurityConfig uses a stateless session policy. Public access is limited to selected authentication operations, public job/employer/analytics GET routes, similar-job retrieval, health/metrics/OpenAPI routes, and email-action redemption. Candidate routes cover CV, matching, candidate profile, and applications. Recruiter routes cover owned jobs, candidate discovery, application review, employer profile, and export. Administrative routes require the Administrator role. Automation routes require authentication, with policy ownership resolved in services.

Role checks are necessary but insufficient for resources addressed by UUID. ApplicationService, for example, verifies that a requested CV belongs to the authenticated Candidate and that a Recruiter owns the Job before listing applicants, changing status, or inviting a candidate. The same pattern is required across identifier-based services. Errors are returned through shared security and application error writers so that 401 unauthenticated and 403 forbidden responses remain distinguishable.

The configuration disables CSRF because bearer tokens are used instead of a server-side browser session. CORS origins are loaded from application configuration. Frame denial, Content Security Policy, and HSTS headers are configured. HSTS is effective only when the application is served through HTTPS, so the local HTTP runtime does not itself demonstrate transport security.

JWT request processing



CareerFit IT AutoPilot — thesis diagram

Figure 4.2. JWT authentication and authorization boundaries

4.3 CV Ingestion and Validation

4.3.1 Upload and Manual Creation

CvController provides multipart upload, manual creation, CV listing, details, processing status, default selection, and deletion. CvIngestionService orchestrates the processing pipeline. Uploaded files are validated, stored through StorageService, and represented by a CV entity whose status makes background progress observable. Manual CV data is converted into labeled raw-text sections such as title, seniority, experience, skills, education, projects, certifications, and languages, after which it uses the same normalization and vectorization path.

Table 4.2. CV processing states

State	Meaning	Typical transition
UPLOADED	Metadata and source have been accepted	Upload/manual creation → validation
VALIDATING	File/content and quality checks are executing	Valid input → processing; hard error → failed
PROCESSING	Extraction, normalization, and vector creation are executing	Vector saved → scoring done
SCORING_DONE	CV vector is available and matching can be queried	Matching runs asynchronously against active jobs
FAILED	Processing cannot continue	Failure reason is persisted and audited

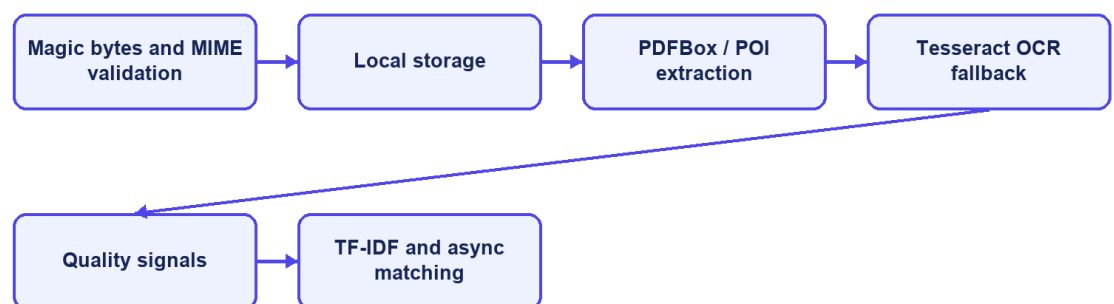
4.3.2 File Validation and Extraction

PdfExtractionService accepts PDF, PNG, JPG/JPEG, and DOCX. It checks the extension, MIME type, and leading magic bytes rather than trusting the filename alone. Empty input and mismatched content are rejected. PDFBox first attempts embedded-text extraction. Apache POI extracts DOCX text. Images use OCR directly. For a PDF with fewer than 50 extracted characters, the service renders up to the configured maximum number of pages and invokes a configurable Tesseract command. The default OCR language set is vie+eng, the default maximum is eight pages, and each OCR process has a timeout.

Text shorter than 50 characters after extraction/OCR is a hard failure. Text between 50 and 199 characters is accepted as sparse and logged as a warning. Encrypted PDFs are rejected. Image dimensions are bounded to reduce excessive memory use, and temporary OCR files are deleted after completion. These controls reduce malformed-input risk, although production deployment would additionally require malware scanning and stronger resource isolation.

QualityValidationService produces structured signals for questionable CV or JD content, including seniority/experience contradictions and salary/content inconsistencies. The design separates blocking errors from warnings so that the user can correct poor-quality data without treating every imperfection as a processing failure.

CV ingestion implementation



CareerFit IT AutoPilot — thesis diagram

Figure 4.3. CV ingestion implementation pipeline

4.4 Text Normalization and TF-IDF Implementation

4.4.1 Text Normalization

TextNormalizationService removes HTML tags, replaces characters outside its English/Vietnamese alphanumeric pattern, lowercases text, splits on whitespace, removes tokens shorter than two characters, and filters a language-specific stop-word set. Language detection is a lightweight heuristic: text containing more than five Vietnamese diacritic characters is classified as Vietnamese; otherwise it is treated as English.

This implementation is deterministic and easy to inspect, but it is not a linguistic word segmenter. Vietnamese multi-syllable skills can be split into independent tokens, and punctuation-containing technology names such as C++, C#, .NET, or Node.js may lose distinguishing characters. These limitations are important when interpreting matching errors and motivate a future domain-aware tokenizer or controlled alias dictionary.

4.4.2 Static Corpus and Vector Construction

TfIdfService builds its IDF map once at application startup from a static seed corpus of representative IT terms. The corpus groups programming languages, frameworks, databases, cloud and DevOps technologies, architecture, testing, security, data/ML, collaboration, seniority, roles, and common Vietnamese IT words. A static corpus prevents scores from shifting whenever a user uploads one CV or adds one Job.

For a token list, term frequency is the token count divided by the total number of tokens. The implemented smoothed IDF is:

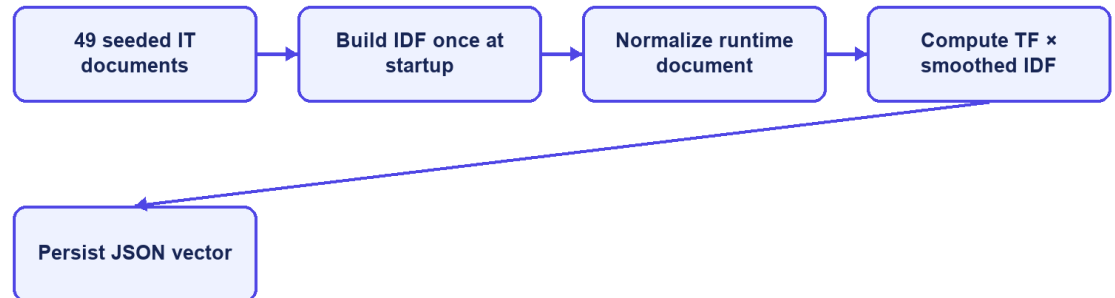
$$idf(t) = \log(1 + N / (1 + df(t))).$$

An unknown term receives $\log(1 + N)$, which treats it as rare and informative. The TF-IDF vector is persisted as JSON and reused during scoring. Cosine similarity iterates over the smaller vector for the dot product, computes both Euclidean magnitudes, and returns zero when either vector is empty.

The unknown-term policy has a practical trade-off. It preserves project-specific technology terms absent from the seed corpus, but a rare misspelling can receive the same high IDF treatment. Input validation, alias normalization, corpus versioning, and

evaluation on realistic data are therefore necessary before treating the score as stable beyond the academic environment.

Static-corpus TF-IDF



CareerFit IT AutoPilot — thesis diagram

Figure 4.4. Seed-corpus initialization and TF-IDF construction

4.5 Matching and Recommendation Implementation

4.5.1 Scoring Service

ScoringService loads the CV vector from extractedTermsJson. For the Job, it prefers learnedProfileVectorJson when a non-empty learned vector exists; otherwise it uses the original tfidfVectorJson. Cosine similarity becomes the raw score, which is multiplied by 100 and rounded to two decimal places. The raw score is stored with six decimal places.

Table 4.3. Implemented score interpretation

Condition	Stored/displayed result
Score below 70	LOW label
Score from 70 to below 90	MEDIUM label
Score at least 90	HIGH label
Score from 35 to below 75 with at least three shared weighted terms	isPotential=true
Score from 35 to below 75 with compatible seniority and at least two shared terms	isPotential=true

Although configuration contains low, medium, and high boundary names, the current assignLabel method uses only the configured medium value (70) and high value (90); the configured low maximum (40) does not create a separate transition. POTENTIAL exists in the enum and schema, but current scoring returns LOW,

MEDIUM, or HIGH and represents potential primarily through the separate boolean and reason. Chapter 5 must test these exact boundaries rather than infer behavior from configuration names.

Match reasons are the five highest-weight Job terms also present in the CV, with Job domain inserted first when available. The potential reason is a fixed explanation about transferable skills and career progression. These reasons are grounded in shared vector terms, but the potential explanation is heuristic rather than a learned causal explanation.

4.5.2 Matching Orchestration and Persistence

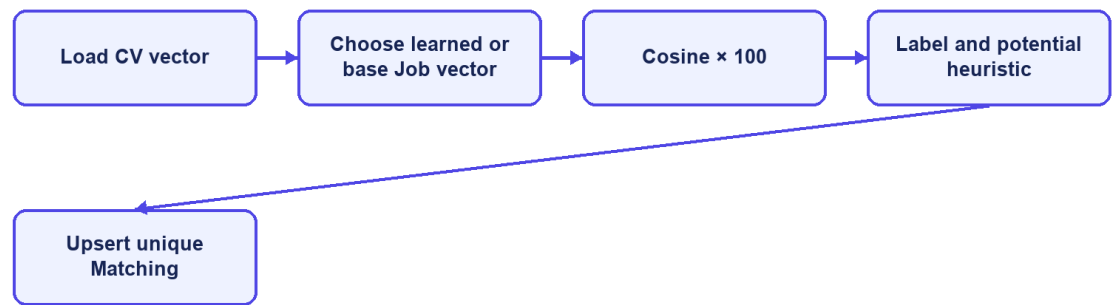
After CV vectorization commits, `AfterCommitExecutor` submits `MatchingService.scoreAllJobsForCv` by CV identifier to the bounded task executor. The service reloads the persisted CV, obtains eligible active Jobs, calls `ScoringService` for each pair, and creates or updates the unique Matching row. Reasons and potential metadata are serialized as JSON. Database uniqueness on `(cv_id, job_id)` prevents duplicate pairs, while conflict handling covers concurrent execution. The same boundary is used when a new or updated Job is scored against existing CVs.

When a Recruiter creates or updates a Job, the service builds the Job vector and schedules post-commit scoring. `MatchingQueryService` shapes Candidate cards and recruiter-oriented results, including pagination and metadata for empty, filtered, low-match, or tied-score states. `CandidatePortfolioVisibilityService` adds portfolio data only when the Candidate enabled `showPortfolioAfterApply` and has actually applied or moved beyond an invitation-only state; otherwise the response contains an explicit hidden reason. Stable score ordering remains important because rounded scores can be equal.

4.5.3 Recommendation Service

`RecommendationService` serves personalized Job results and similar-job retrieval. Candidate recommendations use profile/preferences and available matching information, while similar jobs can be retrieved publicly. The API and UI intentionally keep recommendations separate from the persisted application workflow. A recommendation can be viewed, skipped, or used to start an application without becoming an application automatically unless an enabled `AutoFit` policy separately authorizes that action.

Scoring and persistence



CareerFit IT AutoPilot — thesis diagram

Figure 4.5. Scoring and Matching persistence flow

4.6 Feedback Learning with Rocchio

`FeedbackService.submitFeedback` resolves the Matching and authenticated Candidate, verifies that the Candidate owns the associated CV, and then upserts feedback by (matching_id, actor_id). Concurrent uniqueness conflicts are resolved by reloading and updating the existing row. Each event stores actor role, feedback type, and source channel and creates an audit entry. NOT_INTERESTED is stored but does not trigger Rocchio because it can represent preference rather than technical relevance. GOOD_MATCH, POTENTIAL, and BAD_MATCH trigger post-commit Job-vector learning.

`RocchioService` locks the Job for update and always loads the original Job TF-IDF vector as q . Positive examples are CV vectors attached to GOOD_MATCH or POTENTIAL feedback; negative examples come from BAD_MATCH. The service computes term-wise centroids and applies:

$$q_{new} = 1.0q + 0.75 \text{ positive_centroid} - 0.15 \text{ negative_centroid}.$$

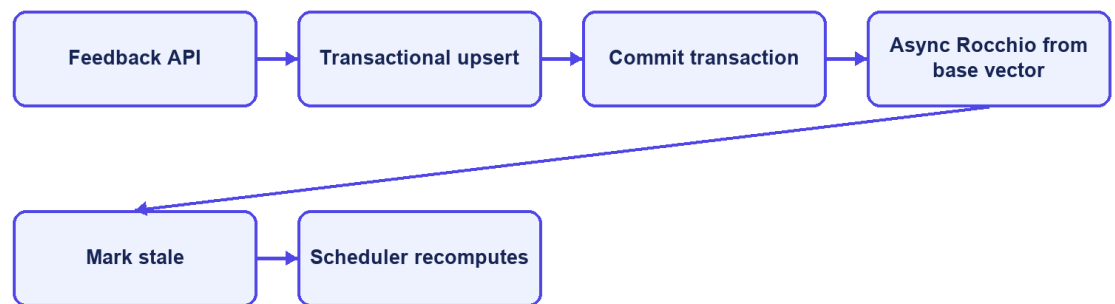
Negative weights are removed rather than persisted. The learned vector is stored in `learnedProfileVectorJson`. Every Matching associated with the Job is then marked `needsRecompute=true`. `AutomationScheduler.recomputeStaleMatchings` rescans these rows every 30 minutes and clears the marker only after successful scoring.

Using the original base vector and the complete feedback history makes recomputation repeatable for the same data and parameters. FeedbackService starts learning after the transaction commits, so Rocchio reads the saved feedback instead of racing the transaction that created it. Job locking prevents two updates from writing separate learned vectors at the same time, and integration tests cover this post-commit behavior.

Table 4.4. Feedback handling

Feedback	Persisted	Rocchio classification	Immediate learning trigger
GOOD_MATCH	Yes	Positive	Yes
POTENTIAL	Yes	Positive	Yes
BAD_MATCH	Yes	Negative	Yes
NOT_INTERESTED	Yes	Neither	No

Feedback implementation



CareerFit IT AutoPilot — thesis diagram

Figure 4.6. Feedback processing and post-commit learning

4.7 Application and Recruiter Workflow

ApplicationService.submit resolves the authenticated Candidate, requires an active Job, rejects an existing Candidate–Job application, and resolves either the requested CV or the Candidate's default CV. If the corresponding Matching exists, it is attached to preserve the score context at application time. saveAndFlush exposes uniqueness conflicts within the service transaction, which are converted into an HTTP conflict response. The service writes an audit event and requests Candidate and Recruiter notifications.